

## Unrestricted Grammars

Type 0 or unrestricted grammars are the most general class of grammars in the Chomsky hierarchy. This grammar can generate arbitrary recursively enumerable languages.

An unrestricted grammar  $G$  is a four-tuple,  $G = (\Sigma, N, S, P)$  with the symbols having their usual meanings, and  $P$  is a finite set of productions of the form  $\alpha \rightarrow \beta$ , for any  $\alpha, \beta \in (N \cup \Sigma)^*$ , with  $\alpha$  containing at least one non terminal symbol.

As usual,  $L(G) = \{w \in \Sigma^* \mid S \xrightarrow{*}_G w\}$ .

**Example 1** The following grammar generates the language  $L_1 = \{a^{2^n} \mid n \geq 0\}$ :

$$\begin{aligned} S &\rightarrow TaU \\ U &\rightarrow AU \mid \varepsilon \\ aA &\rightarrow Aaa \\ TA &\rightarrow T \\ T &\rightarrow \varepsilon \end{aligned}$$

A derivation of the string  $a^8$  using the above grammar is as follows:

$$\begin{aligned} S &\rightarrow TaU \rightarrow TaAU \rightarrow TaAAU \rightarrow TaAAAU \\ &\rightarrow TAaaAAU \rightarrow TAaAaaAU \rightarrow TAAaaaaAU \\ &\rightarrow TAAaaaaAaaU \rightarrow TAAaaaAaaaaU \rightarrow TAAaAaaaaaaaU \\ &\rightarrow TAAAaaaaaaaaaU \xrightarrow{*} aaaaaaaaaa \end{aligned}$$

□

**Example 2** The following grammar generates the language  $L_2 = \{a^n b^n c^n \mid n \geq 0\}$ .

$$\begin{aligned} S &\rightarrow UT \\ U &\rightarrow aUbC \mid \varepsilon \\ Cb &\rightarrow bC \\ CT &\rightarrow Tc \\ T &\rightarrow \varepsilon \end{aligned}$$

For example, the string  $a^2 b^2 c^2$  can be derived as:

$$\begin{aligned} S &\rightarrow UT \rightarrow aUbCT \rightarrow aaUbCbCT \rightarrow aabCbCT \rightarrow aabbCCT \\ &\rightarrow aabbCTc \rightarrow aabbTcc \rightarrow aabbcc \end{aligned}$$

□

We will now show, that the class of languages generated by Type 0 grammars are exactly the Turing recognisable (recursively enumerable) languages.

**Theorem X.1** *If  $L = L(G)$  for some unrestricted grammar  $G$ , then  $L$  is r.e.*

**Proof:** Let us construct a four-tape nondeterministic Turing machine to recognise  $L$ .

We can non-deterministically simulate the grammar  $G$  and try to derive the string  $x$ . If we successfully do so, we accept. The first tape contains the input (and is never changed), the second tape contains the sentential form in the derivation process. The third and fourth tape are used to store  $\alpha$  and  $\beta$  for the production  $\alpha \rightarrow \beta$  being used.

$M$  on input  $x$  first writes  $S$  on the second tape, then loops:

- Non-deterministically choose a position on the second tape.
- Non-deterministically choose a production rule  $\alpha \rightarrow \beta$ .
- Write  $\alpha$  to tape 3 and  $\beta$  to tape 4.
- Compare tape 2 and tape 3 starting from the chosen position in tape 2.
- If the comparison succeeds, replace  $\alpha$  with  $\beta$  in tape 2, shifting the contents if required.
- Compare tape 1 and tape 2, if they are identical then accept.  $\square$

**Theorem X.2** *If  $L$  is r.e., then  $L = L(G)$  for some unrestricted grammar  $G$ .*

**Proof:** Let  $M$  be a DTM recognising  $L$ . WLOG, assume that  $M$  uses a right end marker ( $\vdash$ ) as its rightmost non-blank symbol on its tape throughout its working. Also, modify  $M$  so that it erases the tape contents when it accepts a string, so that the final accepting configuration is always  $\vdash t' \vdash$ , where  $t'$  is the new accepting state.

$G$  will simulate the working of the Turing machine in reverse. If the following represents the sequence of configurations reached by the TM to accept  $x$ :

$$s \vdash x \vdash \rightarrow \dots \rightarrow C_i \rightarrow C_j \rightarrow \dots \rightarrow \vdash t' \vdash$$

Then, the grammar will derive  $x$  by the following derivation:

$$S \rightarrow \vdash t' \vdash \rightarrow \dots \rightarrow C_j \rightarrow C_i \rightarrow \dots \rightarrow s \vdash x \vdash \rightarrow x$$

$M$  could have transitioned from configuration  $C_i$  to  $C_j$  by a transition:

- $\delta(p, a) = (q, b, L)$

$$\dots xpay \dots \rightarrow \dots qxb y \dots$$

- or,  $\delta(p, a) = (q, b, R)$

$$\dots xpay \dots \rightarrow \dots xbqy \dots$$

So, we can define our grammar as  $G = (\Sigma, V = Q \cup (\Gamma \setminus \Sigma) \cup \{S\}, S, P)$  with production rules

$$\begin{aligned} S &\rightarrow \vdash t' \vdash \\ s \vdash &\rightarrow \varepsilon \\ \vdash &\rightarrow \varepsilon \end{aligned}$$

For every transition  $\delta(p, a) = (q, b, L)$ ,  $\forall x \in \Gamma$ ,

$$qxb \rightarrow xpa$$

For every transition  $\delta(p, a) = (q, b, R)$ ,

$$bq \rightarrow pa$$

The equivalence of languages can be shown by induction.  $\square$